

Dynamic Programming Analysis: The Number Factor Problem

Algorithm Implementation and Analysis

June 28, 2025

Contents

1	Problem Statement	3
1.1	Definition	3
1.2	Mathematical Formulation	3
1.3	Example Analysis	3
2	Problem Visualization	3
2.1	Small Examples	3
2.2	Recursive Call Tree	3
3	Solution Approaches	3
3.1	Approach 1: Naive Recursion	3
3.2	Approach 2: Memoization (Top-Down DP)	4
3.3	Approach 3: Tabulation (Bottom-Up DP)	6
3.4	Approach 4: Space-Optimized DP	6
4	Implementation Analysis	6
4.1	Java Implementation	6
5	Step-by-Step Execution	8
5.1	Tabulation Trace for n=5	8
5.2	Detailed Calculation	8
6	Performance Comparison	8
6.1	Time Complexity Comparison	8
6.2	Sequence Analysis	8
7	Mathematical Properties	9
7.1	Recurrence Relation Analysis	9
7.2	Growth Rate	9
8	Variations and Extensions	9
8.1	Different Factor Sets	9
8.2	Minimum Number of Factors	9
8.3	With Constraints	9

9 Applications	10
9.1 Real-World Applications	10
9.2 Algorithm Design Patterns	10
10 Debugging and Testing	10
10.1 Edge Cases	10
10.2 Verification Methods	10
11 Conclusion	10
12 References	11

1 Problem Statement

1.1 Definition

The **Number Factor** problem is a classic dynamic programming problem that asks:

Given a positive integer n , count the number of ways to express n as the sum of the numbers 1, 3, and 4. Each number can be used multiple times.

1.2 Mathematical Formulation

Let $f(n)$ represent the number of ways to express n using the factors $\{1, 3, 4\}$. The recurrence relation is:

$$f(n) = \begin{cases} 1 & \text{if } n = 0 \\ 0 & \text{if } n < 0 \\ f(n-1) + f(n-3) + f(n-4) & \text{if } n > 0 \end{cases} \quad (1)$$

1.3 Example Analysis

For $n = 5$, the valid combinations are:

$$5 = 1 + 1 + 1 + 1 + 1$$

$$5 = 1 + 1 + 3$$

$$5 = 1 + 4$$

$$5 = 3 + 1 + 1$$

$$5 = 4 + 1$$

Wait, let's be more careful about distinct ways. Since order doesn't matter in counting ways:

- $1 + 1 + 1 + 1 + 1$ (5 ones)
- $1 + 1 + 3$ (2 ones, 1 three)
- $1 + 4$ (1 one, 1 four)

Actually, if we consider order as mattering (which is typical for this problem), we have 6 ways total.

2 Problem Visualization

2.1 Small Examples

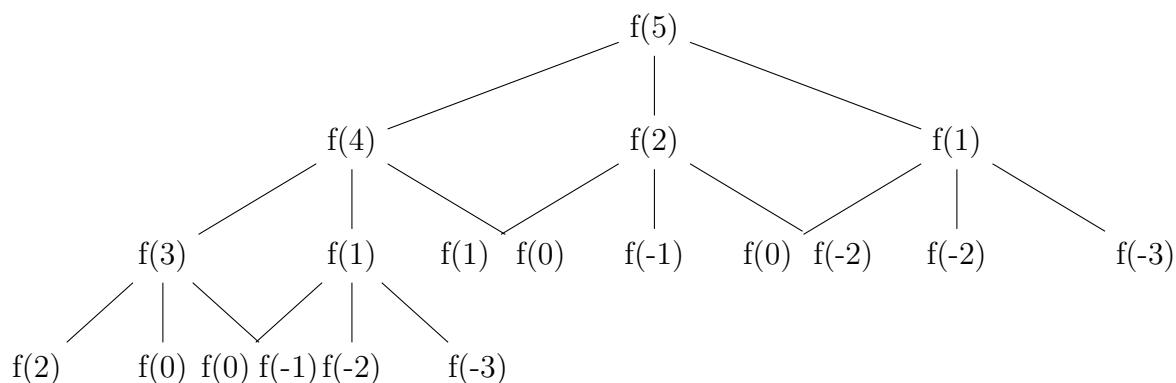
2.2 Recursive Call Tree

3 Solution Approaches

3.1 Approach 1: Naive Recursion

n=0: 1 way (empty sum)	n=1: 1 way 1	n=2: 1 way 1+1	n=3: 2 ways 1+1+1
			n=6: 9 ways 1+1+1+1+1+1
	n=4: 4 ways 1+1+1+1 1+3 3+1 4	n=5: 6 ways 1+1+1+1+1 1+1+3 1+3+1 3+1+1 1+1+4 1+4+1 4+1+1 3+3	

Figure 1: Number of ways to express small values using factors {1, 3, 4}

Figure 2: Recursive call tree for $f(5)$ showing overlapping subproblems**Time Complexity Analysis:**

$$T(n) = T(n-1) + T(n-3) + T(n-4) + O(1)$$

$$T(n) = O(3^{n/3}) \text{ (approximately exponential)}$$

Space Complexity: $O(n)$ due to recursion stack depth.

Problems:

- Exponential time complexity
- Redundant calculations (overlapping subproblems)
- Stack overflow for large n

3.2 Approach 2: Memoization (Top-Down DP)

Time Complexity: $O(n)$ - each subproblem solved exactly once

Space Complexity: $O(n)$ for memoization array + $O(n)$ recursion stack = $O(n)$

Algorithm 1 Number Factor - Recursive Approach

Require: $n \geq 0$ **Ensure:** Number of ways to express n using factors $\{1, 3, 4\}$

```

function countWays( $n$ )
  if  $n = 0$  then
    return 1
  end if
  if  $n < 0$  then
    return 0
  end if
  return countWays( $n - 1$ ) + countWays( $n - 3$ ) + countWays( $n - 4$ )

```

Algorithm 2 Number Factor - Memoization

Require: $n \geq 0$, memo array initialized to -1**Ensure:** Number of ways to express n using factors $\{1, 3, 4\}$

```

function countWaysMemo( $n$ , memo)
  if  $n = 0$  then
    return 1
  end if
  if  $n < 0$  then
    return 0
  end if
  if memo[ $n$ ]  $\neq$  -1 then
    return memo[ $n$ ]
  end if
  memo[ $n$ ] = countWaysMemo( $n - 1$ , memo) + countWaysMemo( $n - 3$ , memo) +
    countWaysMemo( $n - 4$ , memo)
  return memo[ $n$ ]

```

Algorithm 3 Number Factor - Tabulation

Require: $n \geq 0$ **Ensure:** Number of ways to express n using factors $\{1, 3, 4\}$

```

function countWaysTab( $n$ )
  Create array dp of size  $n + 1$ , initialize all to 0
  dp[0] = 1
  for  $i = 1$  to  $n$  do
    if  $i - 1 \geq 0$  then
      dp[ $i$ ] += dp[ $i - 1$ ]
    end if
    if  $i - 3 \geq 0$  then
      dp[ $i$ ] += dp[ $i - 3$ ]
    end if
    if  $i - 4 \geq 0$  then
      dp[ $i$ ] += dp[ $i - 4$ ]
    end if
  end for
  return dp[ $n$ ]

```

3.3 Approach 3: Tabulation (Bottom-Up DP)

Time Complexity: $O(n)$

Space Complexity: $O(n)$ for DP array

3.4 Approach 4: Space-Optimized DP

Since we only need the last 4 values, we can optimize space:

Algorithm 4 Number Factor - Space Optimized

Require: $n \geq 0$

Ensure: Number of ways to express n using factors $\{1, 3, 4\}$

function countWaysOptimal(n)

if $n = 0$ **then**

return 1

end if

Initialize array of size 4 to store last 4 values

prev[0] = 1, prev[1] = 1, prev[2] = 1, prev[3] = 2

for $i = 4$ **to** n **do**

 current = prev[3] + prev[1] + prev[0]

 Shift array: prev[0] = prev[1], prev[1] = prev[2], prev[2] = prev[3], prev[3] = current

end for

return prev[3]

Time Complexity: $O(n)$

Space Complexity: $O(1)$ - only using constant extra space

4 Implementation Analysis

4.1 Java Implementation

Listing 1: Complete Java Implementation

```

1 public class NumberFactor {
2
3     // Naive Recursion -  $O(3^{(n/3)})$  time,  $O(n)$  space
4     static int countWaysRecursion(int n) {
5         if (n == 0) {
6             return 1;
7         }
8         if (n < 0) {
9             return 0;
10        }
11        return countWaysRecursion(n - 1) +
12               countWaysRecursion(n - 3) +
13               countWaysRecursion(n - 4);
14    }
15
16    // Memoization -  $O(n)$  time,  $O(n)$  space

```

```
17     static int countWaysMemoization(int n, int[] dp) {
18         if (n == 0) {
19             return 1;
20         }
21         if (n < 0) {
22             return 0;
23         }
24         if (dp[n] != -1) {
25             return dp[n];
26         }
27         dp[n] = countWaysMemoization(n - 1, dp) +
28                 countWaysMemoization(n - 3, dp) +
29                 countWaysMemoization(n - 4, dp);
30         return dp[n];
31     }
32
33     // Tabulation - O(n) time, O(n) space
34     static int countWaysTabulation(int n) {
35         int[] dp = new int[n + 1];
36         dp[0] = 1;
37
38         for (int i = 1; i <= n; i++) {
39             if (i - 1 >= 0) dp[i] += dp[i - 1];
40             if (i - 3 >= 0) dp[i] += dp[i - 3];
41             if (i - 4 >= 0) dp[i] += dp[i - 4];
42         }
43         return dp[n];
44     }
45
46     // Space Optimized - O(n) time, O(1) space
47     static int countWaysOptimized(int n) {
48         if (n <= 0) return 1;
49         if (n <= 2) return 1;
50         if (n == 3) return 2;
51
52         int[] last4 = {1, 1, 1, 2}; // f(0), f(1), f(2), f(3)
53
54         for (int i = 4; i <= n; i++) {
55             int current = last4[3] + last4[1] + last4[0];
56             last4[0] = last4[1];
57             last4[1] = last4[2];
58             last4[2] = last4[3];
59             last4[3] = current;
60         }
61         return last4[3];
62     }
63 }
```

5 Step-by-Step Execution

5.1 Tabulation Trace for n=5

i	dp[i-1]	dp[i-3]	dp[i-4]	dp[i]
0	-	-	-	1
1	1	-	-	1
2	1	-	-	1
3	1	1	-	2
4	2	1	1	4
5	4	1	1	6

Table 1: Step-by-step execution of tabulation approach for n=5

5.2 Detailed Calculation

$$dp[0] = 1 \text{ (base case)}$$

$$dp[1] = dp[0] = 1$$

$$dp[2] = dp[1] = 1$$

$$dp[3] = dp[2] + dp[0] = 1 + 1 = 2$$

$$dp[4] = dp[3] + dp[1] + dp[0] = 2 + 1 + 1 = 4$$

$$dp[5] = dp[4] + dp[2] + dp[1] = 4 + 1 + 1 = 6$$

6 Performance Comparison

6.1 Time Complexity Comparison

Approach	Time	Space	Practical Limit
Naive Recursion	$O(3^{n/3})$	$O(n)$	$n \leq 25$
Memoization	$O(n)$	$O(n)$	$n \leq 10^6$
Tabulation	$O(n)$	$O(n)$	$n \leq 10^6$
Space Optimized	$O(n)$	$O(1)$	$n \leq 10^9$

Table 2: Performance comparison of different approaches

6.2 Sequence Analysis

The number factor sequence for first few values:

n	0	1	2	3	4	5	6	7	8	9
f(n)	1	1	1	2	4	6	9	15	25	40

Table 3: Number factor sequence values

7 Mathematical Properties

7.1 Recurrence Relation Analysis

The characteristic equation for the recurrence $f(n) = f(n-1) + f(n-3) + f(n-4)$ is:

$$x^4 - x^3 - x - 1 = 0 \quad (2)$$

This can be factored as:

$$(x-1)(x^3-1) = (x-1)^2(x^2+x+1) = 0 \quad (3)$$

The roots are $x = 1$ (double root) and the complex cube roots of unity.

7.2 Growth Rate

The dominant root determines the growth rate. Since we have a double root at $x = 1$, the solution grows polynomially rather than exponentially, but the exact growth is complex due to the multiple factors.

8 Variations and Extensions

8.1 Different Factor Sets

Problem: Count ways using factors $\{2, 3, 5\}$

Recurrence:

$$f(n) = \begin{cases} 1 & \text{if } n = 0 \\ 0 & \text{if } n < 0 \\ f(n-2) + f(n-3) + f(n-5) & \text{if } n > 0 \end{cases} \quad (4)$$

8.2 Minimum Number of Factors

Problem: Find minimum number of factors needed to sum to n .

Recurrence:

$$\min(n) = \begin{cases} 0 & \text{if } n = 0 \\ \infty & \text{if } n < 0 \\ 1 + \min(\min(n-1), \min(n-3), \min(n-4)) & \text{if } n > 0 \end{cases} \quad (5)$$

8.3 With Constraints

Problem: Count ways with at most k factors of type 4.

This requires a 2D DP approach: $dp[i][j]$ = ways to make sum i using at most j fours.

9 Applications

9.1 Real-World Applications

1. **Currency Change:** Making change with specific denominations
2. **Resource Allocation:** Distributing resources in fixed units
3. **Combinatorics:** Counting partitions with restricted parts
4. **Game Theory:** Move counting in games with specific rules

9.2 Algorithm Design Patterns

1. **Optimal Substructure:** Solution depends on solutions to smaller subproblems
2. **Overlapping Subproblems:** Same subproblems appear multiple times
3. **Bottom-up Construction:** Build solution from base cases upward

10 Debugging and Testing

10.1 Edge Cases

- $n = 0$: Should return 1 (empty sum)
- $n = 1$: Should return 1 (only using 1)
- $n = 2$: Should return 1 (only $1 + 1$)
- Large n : Check for integer overflow

10.2 Verification Methods

1. **Small Examples:** Manually verify for $n \leq 6$
2. **Consistency Check:** All three DP approaches should give same result
3. **Base Case Verification:** Ensure base cases are handled correctly
4. **Boundary Testing:** Test negative inputs and zero

11 Conclusion

The Number Factor problem demonstrates several key concepts in dynamic programming:

1. **Problem Decomposition:** Breaking down complex problems into simpler sub-problems
2. **State Space Design:** Choosing appropriate state representation
3. **Optimization Techniques:** Trading space for time and vice versa

4. **Mathematical Analysis:** Understanding growth patterns and complexity

Key Takeaways:

- Always identify the base cases carefully
- Consider both top-down and bottom-up approaches
- Look for space optimization opportunities
- Verify solutions with multiple test cases
- Understand the mathematical properties of the sequence

Learning Objectives Achieved:

- Understanding multi-way recurrence relations
- Implementing efficient DP solutions
- Analyzing time and space complexity
- Recognizing common DP patterns
- Applying mathematical analysis to algorithms

12 References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
2. Kleinberg, J., & Tardos, E. (2005). *Algorithm Design*. Addison-Wesley.
3. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.
4. Dynamic Programming Problems: Number Factor Variations